TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Alexander Emiliano Pilligua Sánchez

PARALELO: A

SEMESTRE: Quinto

ACTIVIDAD #1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

Link de Lucidchart: https://lucid.app/lucidchart/e4a5f083-8ba6-4700-8c05-74a715cdd21d/edit?viewport_loc=-18333%2C-10158%2C4349%2C2232%2C0_0&invitationId=inv_c5a11556-b9bf-44b6-8c4c-590ad410c6f1

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad # 1: Estudio de Caso - Sistema de Control de Rescate y Adopción

1. Descripción del problema

En la actualidad, los refugios y organizaciones dedicadas al rescate y protección animal necesitan gestionar de manera organizada los casos de animales que se encuentran en situación de abandono, maltrato, peligro o vulnerabilidad. Sin un sistema informático adecuado, el registro de los animales rescatados, la asignación de voluntarios y veterinarios, el seguimiento de los tratamientos y la administración de los procesos de adopción pueden volverse actividades complejas, dificultando el acceso oportuno a la información y el control de cada caso.

Ante esta problemática, se propone el diseño de un Sistema de Control de Rescate y Adopción Animal, cuyo propósito es centralizar y organizar la información relacionada con los animales rescatados, incluyendo perros, gatos y fauna silvestre. El sistema permitirá registrar nuevos casos de rescate, clasificar a los animales según su especie, generar expedientes individuales, asignar responsables y actualizar los estados del proceso, desde el rescate inicial hasta la recuperación y la posible adopción.

Asimismo, la solución facilitará la comunicación entre los ciudadanos que reportan casos, los voluntarios, los veterinarios y los administradores del refugio, mediante mecanismos de notificación y seguimiento de las actividades realizadas. Para garantizar una estructura organizada, reutilizable y mantenible, el diseño incorporará patrones de diseño de software, principios SOLID y una arquitectura que permita separar las responsabilidades de cada componente. Finalmente, se utilizará Git y la metodología Gitflow para gestionar las versiones del proyecto, registrar los cambios y mantener un historial ordenado del desarrollo.

De esta manera, se busca establecer una solución informática que contribuya a mejorar la organización, la trazabilidad y la eficiencia de los procesos de rescate, atención y adopción animal.

2. Identificación de los protagonistas del sistema

Los protagonistas son las personas que tienen interacción directa con el Sistema de Control de Rescate y Adopción Animal. Cada protagonista tendrá roles y permisos particulares, según las tareas que realicen dentro del proceso de rescate, atención veterinaria y adopción.

Para el funcionamiento del sistema, se destacan los siguientes protagonistas principales:

2.1. Ciudadano o reportador

Es la persona que identifica a un animal en condiciones de abandono, maltrato, peligro o vulnerabilidad y reporta el caso al refugio. Su participación es fundamental para iniciar el proceso de rescate mediante el registro de información relevante, como la

localización del animal, su especie, una descripción de su estado y las observaciones disponibles.

Funciones principales:

- Registrar denuncias de posibles casos de rescate.
- Proporcionar información sobre la ubicación y las condiciones del animal.
- Consultar el estado de los reportes, si el sistema permite esta funcionalidad.

2.2. Voluntario

Es la persona encargada de ayudar en las actividades de rescate y cuidado de los animales. A través del sistema, podrá revisar los casos que le han sido asignados, documentar las acciones realizadas y actualizar la información relacionada con el seguimiento del rescate.

Funciones principales:

- Revisar los casos asignados.
- Participar en las actividades de rescate y traslado.
- Registrar observaciones sobre el estado del animal.
- Actualizar el seguimiento de las tareas bajo su responsabilidad.

2.3. Veterinario

Es el profesional encargado de evaluar la salud de los animales rescatados y proporcionar la atención veterinaria necesaria. Su intervención permite mantener un registro organizado de los diagnósticos, tratamientos y evolución de cada animal.

Funciones principales:

- Consultar los expedientes de los animales asignados.
- Registrar evaluaciones y diagnósticos veterinarios.
- Documentar tratamientos y observaciones clínicas.
- Informar sobre la evolución y recuperación del animal.

2.4. Administrador del refugio

Es la persona responsable de supervisar el funcionamiento general del sistema y coordinar las actividades relacionadas con los casos de rescate. Tiene la autoridad para gestionar los expedientes, asignar responsabilidades entre el personal y controlar las etapas del proceso.

Funciones principales:

- Gestionar los expedientes de los animales.
- Asignar voluntarios y veterinarios a los casos.
- Supervisar y actualizar el estado de los procesos.
- Administrar las solicitudes y registros de adopción.

- Consultar el historial de actividades y los cambios realizados.

2.5. Resumen de los actores

Tabla 1

Actor	Responsabilidad Principal	Nivel de Acceso
Ciudadano o representante	Comunicar casos de rescate	Registro y consulta de sus reportes, según los permisos definidos.
Voluntario	Colaborar en el rescate y seguimiento	Consulta y actualización de casos asignados.
Veterinario	Gestionar la atención y los registros clínicos	Acceso a expedientes y actualización de información veterinaria.
Administrador	Coordinar y supervisar los procesos	Gestión administrativa de los casos y las adopciones.

Nota: *Los permisos específicos se establecerán durante el diseño del sistema. Esta separación permite aplicar el principio de mínimo privilegio y evita que todos los usuarios puedan modificar cualquier información.*

3. Requerimientos del sistema

Los requerimientos del Sistema de Control de Rescate y Adopción Animal establecen las funcionalidades y las condiciones que deberá cumplir la solución informática para gestionar adecuadamente los casos de rescate. Estos se dividen en requerimientos funcionales, que describen las operaciones que podrá realizar el sistema, y requerimientos no funcionales, que determinan las características de calidad, seguridad y mantenimiento que deberá ofrecer.

3.1. Requerimientos funcionales

Los requerimientos funcionales especifican las acciones que el sistema deberá permitir a los ciudadanos, voluntarios, veterinarios y administradores del refugio.

Tabla 2

Código	Requerimiento funcional	Descripción
RF01	Registras caso de rescate	Permitir el ingreso de reportes con información del animal, ubicación y descripción del caso.
RF02	Clasificar animales	Categorizar los animales según su especie: perro, gato o fauna silvestre.

RF03	Gestionar expedientes	Crear y consultar un expediente individual para cada animal rescatado.
RF04	Asignar responsables	Permitir al administrador asignar voluntarios o veterinarios a cada caso.
RF05	Gestionar estados	Actualizar el estado del caso según su evolución: rescatado, en tratamiento, en recuperación, disponible para adopción o adoptado.
RF06	Registrar atención veterinaria	Guardar diagnósticos, tratamientos y observaciones relacionadas con la salud del animal.
RF07	Generar notificaciones	Notificar a los responsables cuando se registre un caso o se les asigne una nueva actividad.
RF08	Gestionar adopciones	Registrar solicitudes de adopción y documentar su aprobación o rechazo.
RF09	Consultar historial	Mantener un registro de las asignaciones, actualizaciones y actividades realizadas en cada caso.
RF10	Gestionar usuarios	Permitir al administrador gestionar usuarios y establecer sus permisos de acceso.

3.2. Requerimientos no funcionales

Los requerimientos no funcionales establecen las condiciones de calidad que deberá cumplir el sistema para funcionar de manera segura, organizada y eficiente.

Tabla 3

Código	Requerimiento no funcional	Descripción
RNF01	Mantenibilidad	El código deberá organizarse en módulos con responsabilidades claramente definidas para facilitar su modificación y mantenimiento.
RNF02	Escalabilidad	La arquitectura deberá permitir incorporar nuevas especies, funcionalidades y usuarios sin modificar completamente el sistema.
RNF03	Seguridad	El acceso a las funciones y los datos deberá controlarse según el rol de cada usuario.
RNF04	Trazabilidad	El sistema deberá conservar un historial de las modificaciones relevantes y el proyecto deberá mantener un registro de versiones mediante Git.
RNF05	Reutilización	Se deberán aplicar patrones de diseño y componentes reutilizables para reducir la duplicación de código.
RNF06	Fiabilidad	El sistema deberá validar la información ingresada y gestionar los errores para evitar registros inconsistentes.
RNF07	Usabilidad	La interfaz deberá ser comprensible y permitir que los usuarios realicen sus tareas de manera sencilla.
RNF08	Integridad de los datos	Los expedientes, las asignaciones y los registros de adopción

	deberán mantener relaciones coherentes y evitar información contradictoria.

3.3. Relación entre los requerimientos y los patrones de diseño

Los requerimientos identificados orientan la selección de los patrones de diseño que se utilizarán en la arquitectura del sistema.

Tabla 4

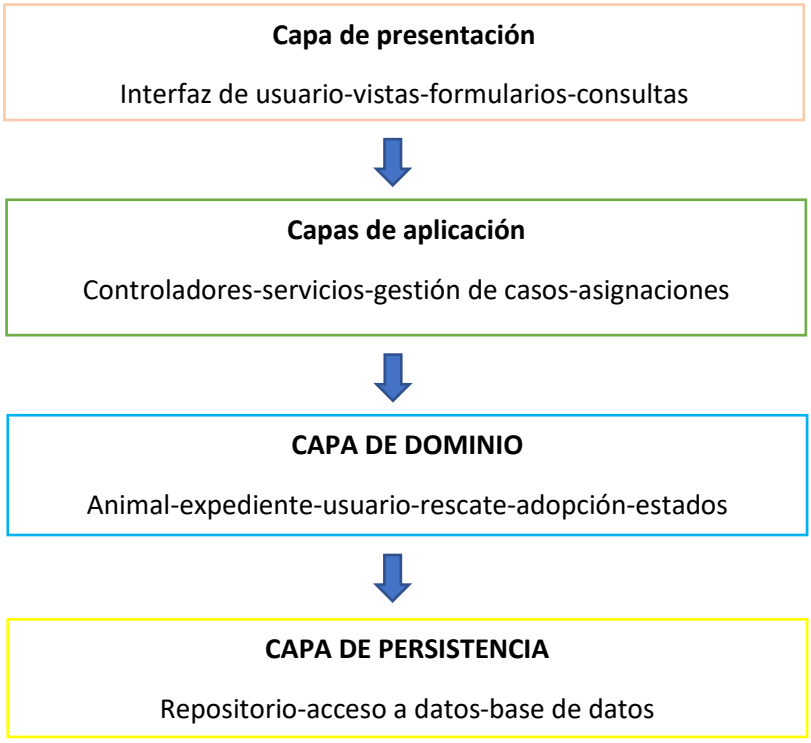
Necesidad identificada	Patrón	Justificación
Centralizar un servicio de gestión de casos.	Singleton	Controla la creación de una instancia única del servicio dentro del ámbito definido.
Informar automáticamente sobre nuevos casos y asignaciones.	Observer	Permite notificar a los componentes suscritos cuando ocurren eventos relevantes.
Crear expedientes según la especie del animal.	Factory Method	Organiza la creación de objetos específicos mediante una estructura extensible.
Separar la interfaz de las reglas del negocio.	MVC	Distribuye las responsabilidades entre Modelo, Vista y Controlador.

4. Diseño de la arquitectura lógica del sistema

La arquitectura lógica del Sistema de Control de Rescate y Adopción Animal define la organización de sus componentes, las responsabilidades de cada módulo y la forma en que estos interactúan para procesar la información. Su diseño busca separar las funciones de presentación, la lógica del negocio y el acceso a los datos, facilitando el mantenimiento, la reutilización y la ampliación futura del sistema.

Para cumplir con estos objetivos, se propone una arquitectura por capas que incorpore el patrón arquitectónico MVC (Modelo-Vista-Controlador) y los patrones de diseño Singleton, Observer y Factory Method.

4.1. Organización de las capas



Nota. *Elaboración propia.*

El esquema representa las cuatro capas propuestas y sus principales responsabilidades. Las flechas indican la comunicación entre ellas, aunque la implementación deberá respetar las dependencias entre componentes y evitar que la interfaz acceda directamente a la base de datos.

4.2. Descripción de las capas

4.2.1. Capa de presentación

Es la parte del sistema con la que interactúan los usuarios. Proporciona formularios y pantallas para registrar reportes, consultar expedientes, asignar responsables y gestionar adopciones.

Sus funciones principales son:

- Presentar la información de los animales y los casos de rescate.
- Recibir los datos ingresados por los usuarios.
- Mostrar los resultados de las operaciones y los mensajes de validación.
- Facilitar la navegación según los permisos de cada usuario.

Esta capa corresponde principalmente a la Vista del patrón MVC.

4.2.2. Capa de aplicación

Coordina las operaciones solicitadas por los usuarios y comunica los componentes necesarios para ejecutar cada proceso.

- Sus funciones principales son:
- Procesar los reportes de rescate.
- Coordinar la creación de expedientes.
- Gestionar la asignación de voluntarios y veterinarios.
- Coordinar los cambios de estado de los casos.
- Solicitar el envío de notificaciones cuando ocurran eventos relevantes.

En esta capa se ubican los Controladores de MVC y los servicios de aplicación. Estos componentes coordinan las operaciones sin concentrar todas las reglas del negocio en una sola clase.

4.2.3. Capa de dominio

Contiene las entidades, los objetos y las reglas fundamentales del negocio. Representa los elementos que intervienen en el proceso de rescate y adopción animal.

Entre sus principales elementos se encuentran:

- Animal: representa al animal registrado.
- Expediente: contiene la información del caso de rescate.
- Usuario: representa a las personas que utilizan el sistema.
- Voluntario: representa al personal que participa en el rescate.
- Veterinario: representa al profesional encargado de la atención clínica.
- Adopción: registra la información del proceso de adopción.

En esta capa se aplicará el patrón Factory Method para organizar la creación de expedientes específicos según la especie, y se definirán los eventos que podrán activar las notificaciones mediante Observer.

4.2.4. Capa de persistencia

Es responsable de almacenar, consultar y actualizar la información del sistema mediante mecanismos de acceso a datos.

Sus funciones principales son:

- Guardar los expedientes y los datos de los animales.
- Consultar los casos registrados.
- Actualizar los estados y las asignaciones.
- Conservar los registros de atención veterinaria y adopción.
- Facilitar la integridad y la consistencia de los datos.

Se propone utilizar el patrón Repository para separar la lógica del negocio de los detalles específicos de almacenamiento. La tecnología concreta de base de datos se seleccionará durante una etapa posterior de implementación.

4.3. Integración de los patrones de diseño

Los patrones se incorporarán en los componentes donde aporten una solución concreta a los requerimientos identificados.

Tabla 5

Patrón	Componentes compuestos	Función dentro del sistema
Singleton	Servicio central de gestión de casos o configuración	Proporcionar una instancia compartida dentro del ámbito definido.
Observer	Servicio de eventos y notificaciones	Informar a los observadores suscritos sobre nuevos rescates o asignaciones.
Factory Method	Fábrica de expedientes	Crear expedientes especializados para perros, gatos y fauna silvestre.
MVC	Presentación y control de solicitudes	Separar la interfaz, la coordinación de solicitudes y los datos del dominio.
Repository	Persistencia	Abstraer las operaciones de almacenamiento y consulta.

El patrón Repository se incorpora como una propuesta complementaria para reforzar la separación de responsabilidades. Los cuatro patrones principales solicitados se mantendrán como eje de la documentación.

Consideración técnica: Singleton no debe utilizarse automáticamente para todas las conexiones a datos. En una aplicación escalable, el manejo de conexiones suele delegarse a un pool de conexiones o a un contenedor de dependencias. Para este diseño académico, se aplicará Singleton únicamente a un servicio cuya instancia única tenga una justificación clara.

4.4. Flujo general de funcionamiento

El funcionamiento propuesto comienza cuando un ciudadano registra un reporte de un animal que necesita ayuda. El controlador recibe la solicitud y coordina la validación de los datos. Posteriormente, la fábrica correspondiente crea el expediente según la especie registrada y el servicio de gestión guarda la información mediante la capa de persistencia.

Una vez registrado el caso, el sistema genera un evento que permite notificar a los responsables correspondientes. El administrador puede asignar un voluntario o veterinario y, durante el seguimiento, los usuarios autorizados actualizan la información del rescate, los tratamientos y los estados del expediente hasta llegar a una posible adopción.

Este flujo permite que cada componente desempeñe una función específica y que los cambios en una parte del sistema afecten lo menos posible a las demás.

5. Diseño del diagrama de casos de uso UML

5.1. Objetivo del diagrama

El diagrama de casos de uso tiene como finalidad representar las interacciones entre los actores y las funcionalidades del sistema. Permite identificar las operaciones principales que realizará cada usuario, establecer los límites del sistema y visualizar las responsabilidades de los participantes en los procesos de rescate, atención veterinaria y adopción.

5.2. Elementos que debe incluir

El diagrama tendrá cuatro actores principales:

- Ciudadano o reportante: registra reportes de animales que necesitan ayuda y consulta el estado de sus reportes.
- Voluntario: consulta los casos asignados y registra las actividades de rescate.
- Veterinario: consulta expedientes, registra diagnósticos y documenta tratamientos.
- Administrador del refugio: gestiona expedientes, asigna responsables, supervisa los estados y administra las adopciones.

Dentro del límite del sistema colocaremos los siguientes casos de uso:

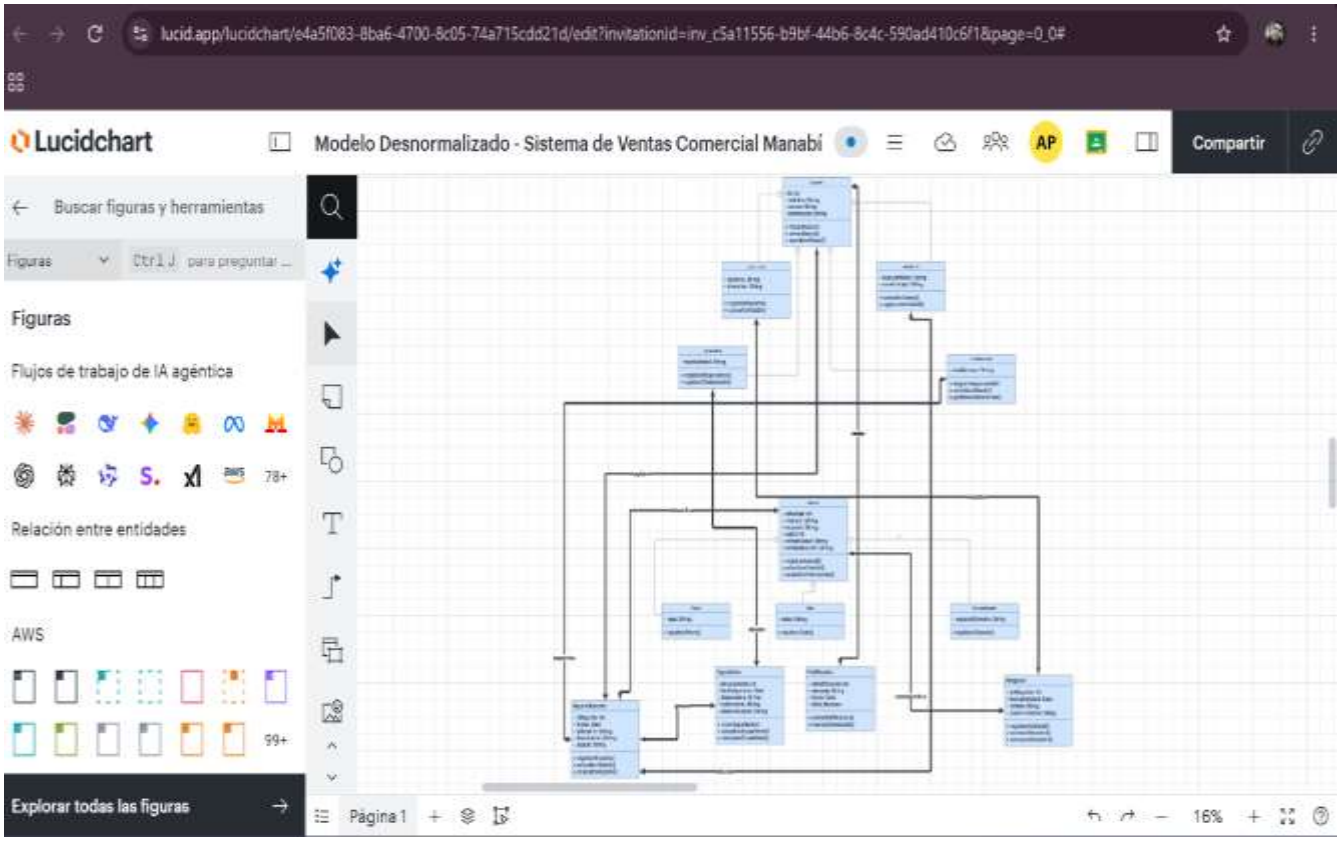
Tabla 6

Código	Caso de uso	Actor principal
CU01	Registrar reporte de rescate	Ciudadano
CU02	Consultar estado del reporte	Ciudadano
CU03	Consultar casos asignados	Voluntario
CU04	Registrar actividad de rescate	Voluntario
CU05	Consultar expediente animal	Veterinario
CU06	Registrar diagnóstico y tratamiento	Veterinario

CU07	Crear expediente animal	Administrador
CU08	Asignar voluntario o veterinario	Administrador
CU08	Actualizar estado del caso	Administrador
CU10	Gestionar solicitudes de adopción	Administrador
CU11	Consultar historial del caso	administrador

5.3. Diagrama de casos de uso

Este es el diseño que utilizaremos como base para elaborar el diagrama. Representa los actores, las funcionalidades y las notificaciones automáticas del sistema.



Descripción del diagrama de clases UML. El diagrama representa la estructura estática del Sistema de Rescate Animal mediante clases que gestionan usuarios, animales, reportes de rescate, expedientes, notificaciones y adopciones. Las relaciones de herencia permiten especializar los usuarios y los tipos de animales, mientras que las asociaciones representan las interacciones entre los componentes del sistema. Este modelo facilita la organización de la información y define la estructura básica para el desarrollo de la aplicación.

5.4. Descripción de las relaciones

1. Ciudadano o reportante: puede registrar un reporte de rescate y consultar su estado.

2. Voluntario: puede consultar los casos asignados y registrar las actividades realizadas.

3. Veterinario: puede consultar los expedientes de los animales bajo su responsabilidad y registrar los diagnósticos y tratamientos.

4. Administrador: puede crear expedientes, asignar responsables, actualizar estados, gestionar adopciones y consultar el historial.

5. Notificaciones automáticas: el sistema contempla notificaciones cuando se registra un reporte o se realiza una asignación. En el diseño detallado, estas operaciones se relacionarán con el patrón Observer.

La relación <<include>> indica que una funcionalidad incorpora obligatoriamente otra durante su ejecución. En este diseño, el envío de notificaciones se plantea como parte del registro de reportes y de la asignación de responsables.

5.5. Conclusión del diagrama de casos de uso

El diagrama permite identificar las funcionalidades principales del sistema y las responsabilidades de cada actor. Su elaboración facilita la comprensión del alcance de la solución y establece una base para el diseño de las clases, los métodos y las interacciones internas que se detallarán en los siguientes apartados.

6. Aplicación de patrones de diseño

6.1. Objetivo de los patrones de diseño

Los patrones de diseño permiten organizar el software mediante soluciones reutilizables a problemas comunes de desarrollo. En el Sistema de Rescate Animal se aplicarán patrones que faciliten el mantenimiento, la reutilización del código y la separación de responsabilidades entre los componentes del sistema.

6.2. Patrones de diseño seleccionados

Tabla 7. Patrones de diseño aplicados al Sistema de Rescate Animal

Patrón	Aplicación en el sistema	Beneficio
MVC (Modelo-Vista-Controlador)	Separa la interfaz de usuario, la lógica de control y los datos del sistema.	Facilita el mantenimiento y la organización del software.
Singleton	Permite disponer de una instancia única del servicio central de gestión de casos	Evita instancias duplicadas del servicio central.

	de rescate, cuando sea necesario compartir ese recurso.	
Observer (Observador)	Permite notificar a los usuarios interesados cuando se registra un nuevo caso o se modifica una asignación.	Facilita la comunicación de cambios y las notificaciones.
Factory Method (Método de fábrica)	Permite crear objetos de animales o expedientes según el tipo correspondiente, como perro, gato o fauna silvestre.	Facilita la incorporación de nuevos tipos sin modificar toda la lógica de creación.
Repository (Repositorio)	Centraliza las operaciones de consulta, registro, actualización y recuperación de datos de animales, reportes y adopciones.	Separa el acceso a los datos de la lógica del negocio.

Nota: *Elaboración propia.*

6.3. Justificación de los patrones

La selección de estos patrones responde a las necesidades del Sistema de Rescate Animal:

- MVC: organiza la interfaz, el control de las operaciones y los datos.
- Singleton: permite controlar el acceso a un servicio compartido cuando se requiere una única instancia.
- Observer: facilita la comunicación de cambios relacionados con los reportes y las asignaciones de rescate.
- Factory Method: permite crear diferentes tipos de objetos según sus características.
- Repository: organiza las operaciones de acceso a la información almacenada.

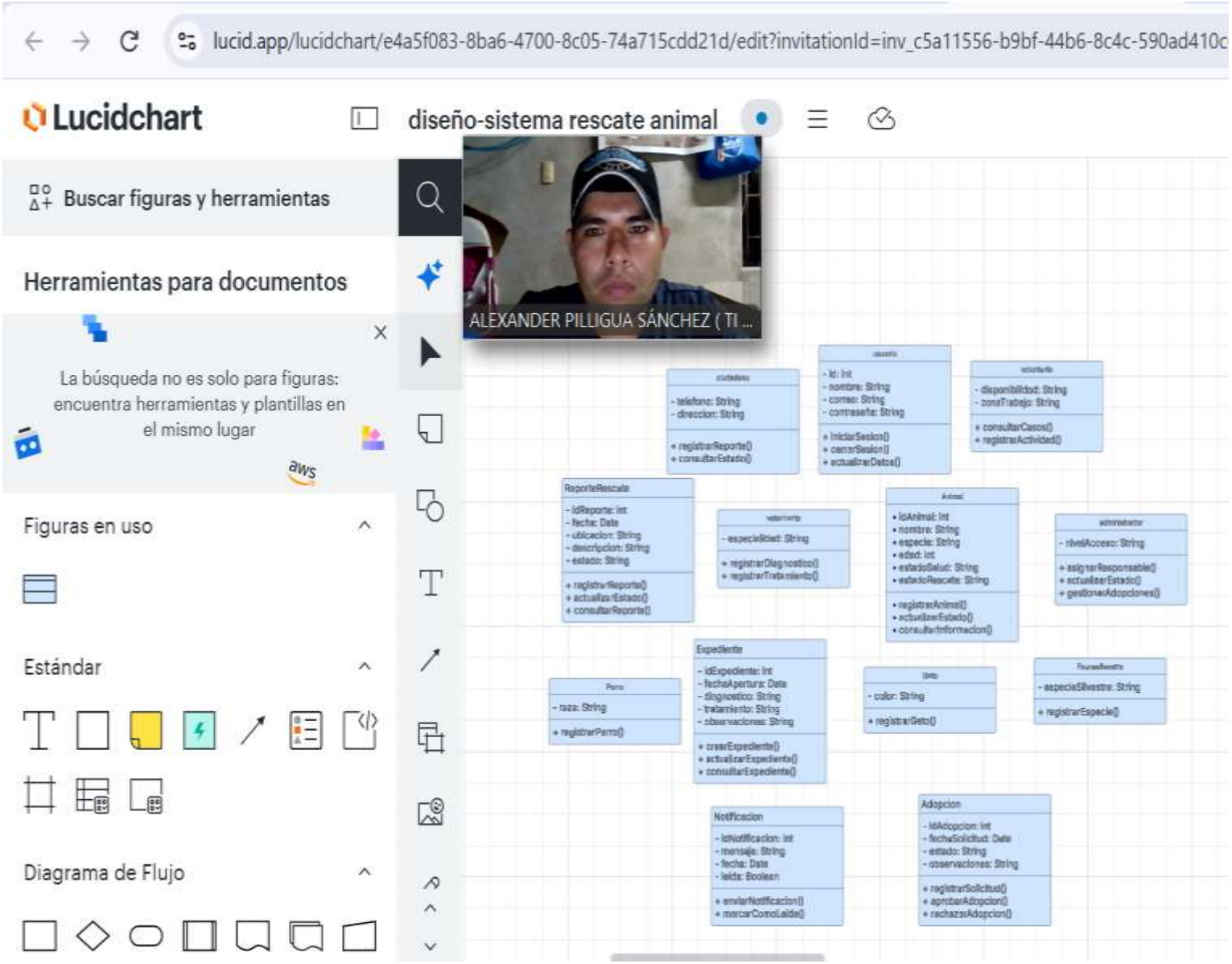
La aplicación conjunta de estos patrones contribuye a obtener una arquitectura modular, mantenible y extensible, que facilita futuras mejoras del sistema.

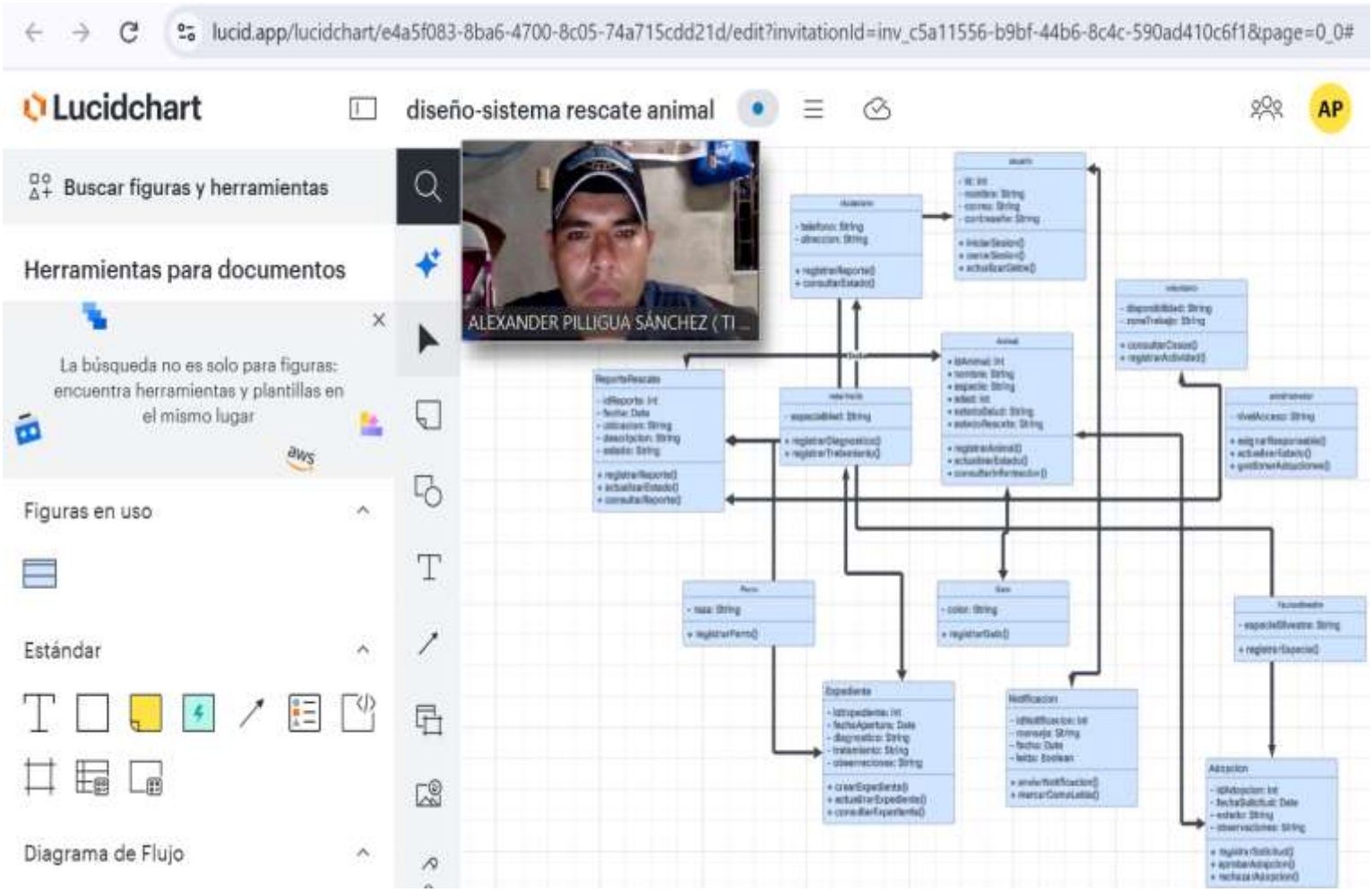
6.4. Relación con los principios SOLID

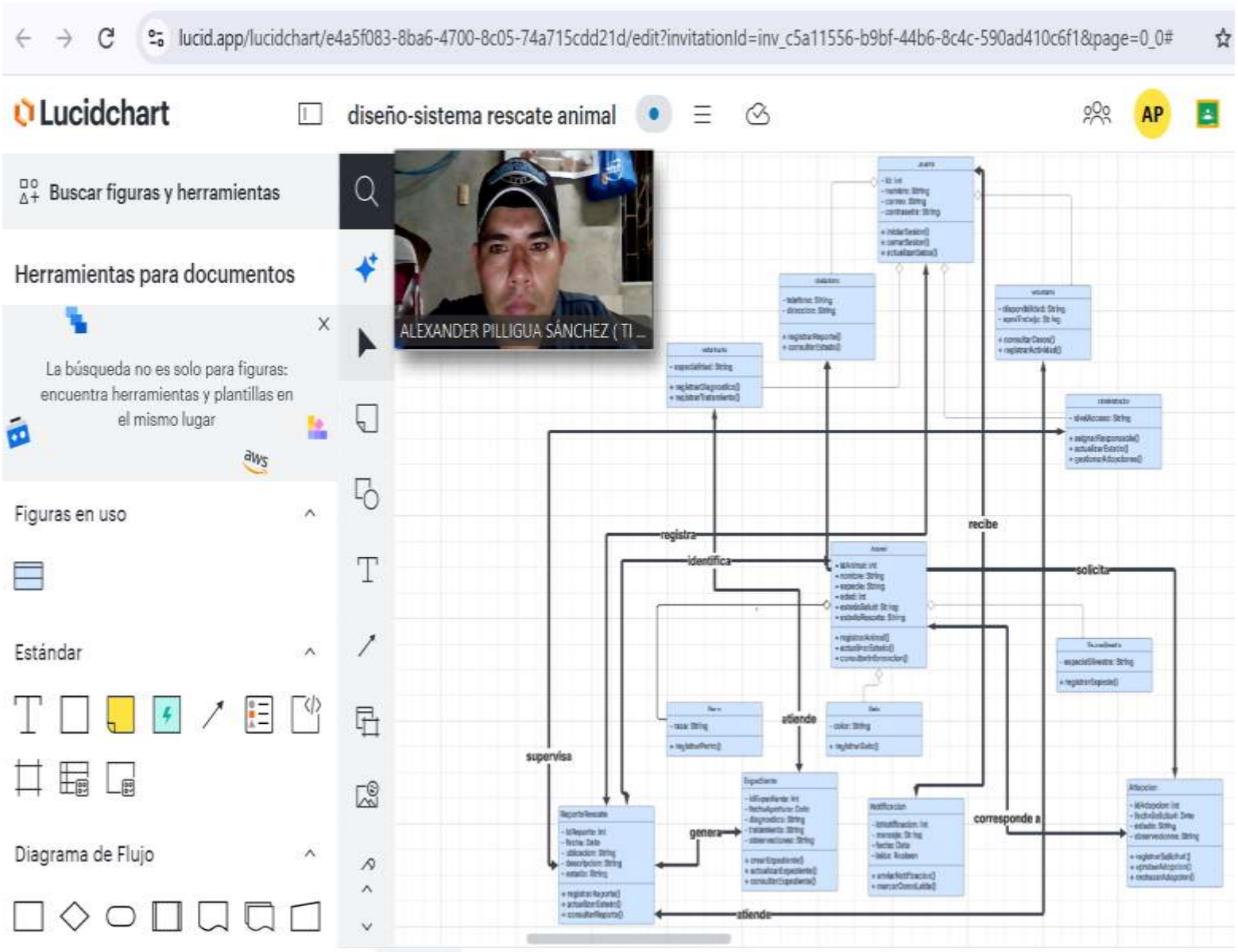
Los patrones seleccionados complementan los principios SOLID previamente considerados en el diseño. MVC y Repository favorecen la separación de responsabilidades; Factory Method facilita la extensión del sistema; y Observer reduce el acoplamiento entre los componentes que generan eventos y los que reciben

notificaciones. Singleton debe utilizarse únicamente cuando exista una necesidad real de compartir una instancia, evitando dependencias globales innecesarias.

Evidencias




lucid.app/lucidchart/e4a5f083-8ba6-4700-8c05-74a715cdd21d/edit?invitationId=inv_c5a11556-b9bf-44b6-8c4c-590ad410c6f1&page=0_0#
Lucidchart
diseño-sistema rescate animal
AP
Buscar figuras y herramientas
Herramientas para documentos
La búsqueda no es solo para figuras: encuentra herramientas y plantillas en el mismo lugar
Figuras en uso
Estándar
Diagrama de Flujo
ALEXANDER PILLIGUA SÁNCHEZ (TI ...



Link de Lucidchat:
https://lucid.app/lucidchart/e4a5f083-8ba6-4700-8c05-74a715cdd21d/edit?viewport_loc=-18333%2C-10158%2C4349%2C2232%2C0_0&invitationId=inv_c5a11556-b9bf-44b6-8c4c-590ad410c6f1

Bibliografía

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.

2. Git. (s. f.). Git documentation. https://git-scm.com/doc

3. GitHub. (s. f.). GitHub flow. https://docs.github.com/en/get-started/using-github/github-flow

4. Martin Fowler. (2002). Patterns of enterprise application architecture. Addison-Wesley.

5. Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), version 2.5.1. https://www.omg.org/spec/UML/2.5.1